CLASSMATE
Date :
Page :

SRS Document

Topic: EatSure

1. Introduction

1.1. Purpose

This document describes the Software requirement Specifications (SRS) for EatSure food delivery app. It defines the functional and non-functional requirements of the system for developers, testers and stakeholders.

1.2. Scope

EatSure is an online food delivering platform that allows user to browse restaurants, order food, make payments and track deliveries. The system ensure quality-checked food from trusted brands and provide a safe, hygenic food experience

The application will -

Allow user to order food online

Provide real-time ordering (track Orders)

Enables secure applications to make payment online

Offer customer support services.

1.3. Definations and Acronyms

UI - User Interface
OTP - One time password
DB - Database
API - Application Programming Interface.

1.4. References - IEEE SRS Documentation Standard (IEEE 830)

2. Overall Description

2.1 Product Perspective

Eatsure is mobile and web-based food ordering platform integrated with:

1. Customer mobile application
2. Restaurant Admin Panel
3. Delivery Partner App
4. Backend server and Database

2.2. Product Functions

Major functions of system include:

- User Registration and Login
- Restaurant and menu browsing
- Food Ordering
- Payment Processing
- Order tracking
- Rating and Reviews

2.3. User Classes

	User Type	Description
1.	Customer	Order food
2	Restaurant Admin	Manage menu and orders
3.	Delivery Partner	Delivers food
4.	System Admin	Manage entire system

2.4. Operating Environment

- Android / ios device
- Web Browser
- Internet connection requirement
- Backend server (cloud-based)
- Database (MySQL / MongoDB)

3. Specific Requirements

3.1. Functional Requirement

FR1: User Registration

Input- User enters name, email, phone number, passw

Process- System validates details and stores user data in database

Output- User account is created successfully and confirmation message displayed.

CLASSMATE
Date :
Page :

FR2. User Login

Input- User enters registered email/phone and password
Process- System verifies credentials with database
Output- User is redirected to home page upon successful login.

FR3. Browse Restaurants

Input- User selects location or enable GPS
Process- System fetches available restaurants based on location.
Output- List of restaurants displayed with ratings and delivery time.

FR4. Search Food Items

Input- User enters food name in search bar.
Process- System matches keyword with menu database
Output- Relevant food items are displayed.

FR5. Add to cart

Input- User selects food item and quantity
Process- System adds selected items to virtual cart
Output- Cart updated with selected items & total price.

FR6. Place Order

Input - User confirms cart and delivery address.
Process - System sends order details to restaurants and calculates final bills.
Output - Order ID generated and order confirmation option.

FR7. Online Payment

Input - User selects payment method (UPI, Card, Wallet)
Process - System connects to payment gateway and processes transaction.
Output - Payment success/fails message displayed.

FR8. Order Tracking

Input - User selects "Track Order".
Process - System retrieves live status from delivery partner system.
Output - Real-time order status displayed on screen

FR9. Ratings and Reviews

Input - User gives rating and written review.
Process - System stores feedback in database
Output - Review visible on restaurant page.

FR10. Order cancellation

Input - User selects cancel option before preparation starts.

Process - System verifies cancellation eligibility.
Output - Order cancelled and refund initiated (if applicable)

FR11.* AI based Personalised Nutrition Recommendation

Input - User enters dietary preferences (eg - weight loss, high-protein, diabetic-friendly) or health goals in profile settings.
Process - System analyses user preferances and Order history using AI algorithms and filter suitable menu items.
Output - Personalised healthy meal recommendations displayed on home screen.

FR12.* Scheduled Smart Group Ordering (Split bill Payment Feature)

Input - User selects option "Group order" Adds multip participants via phone number/link.
Process - System allows each participants to add items separately and automatically splits the total bill.
Output - Individual payment links sent to participants and order placed ~~onced~~ once all payments are completed.

CLASSMATE
Date :
Page :

3.2. Non-Functional Requirements

NFR1. Performance Requirements

(i) System should load within 3 seconds.
(ii) Support atleast 10,000 concurrent users.
(iii) Payment processing should complete within 5 seconds.

NFR2. Security Requirements

(i) User password must be encrypted
(ii) Secure HTTPS communication required.
(iii) Payment data must follow PCI-DSS standards.

NFR3. Usability Requirements

(i) Simple and user-friendly interface.
(ii) Easy navigation with minimal steps.
(iii) App should support English language

NFR4. Reliability Requirements

(i) System shall maintain 99% uptime availability
(ii) order data shall be stored reliably with duplication or loss.
(iii) In case of payment failure, the system shall ensure refund processing without data inconsistency.

NFR5. Scalability Requirements

(i) System shall handle increasing numbers of restaur-ants and users.
(ii) The database shall support large volumes of order

history and transaction data.

NFR6. Compatibility Requirements
1. Application shall be compatible with Android and ios platforms.
2. Web version shall function properly on major browsers (Chrome, Edge, Safari)
3. The system shall integrate with third-party payments gateway and GPS tracking services.

NFR7. Maintainability Requirements
1. The system shall follow modular architecture for easier upgrades.
2. Bugs shall be resolved without affecting live order processing.
3. New restaurants shall be added without down-time.

NFR8. Data Privacy and Compliance Requirements
1. The system shall comply with applicable data protection regulation.
2. User personal information shall not be shared with third party without explicit consent.
3. Users shall have the option to request deletion of personal data from the system.

4. External Interface Requirements.

4.1 User Interface

- Clean Home Screen with search bar
- Cart icon with item count
- Order tracking map interface

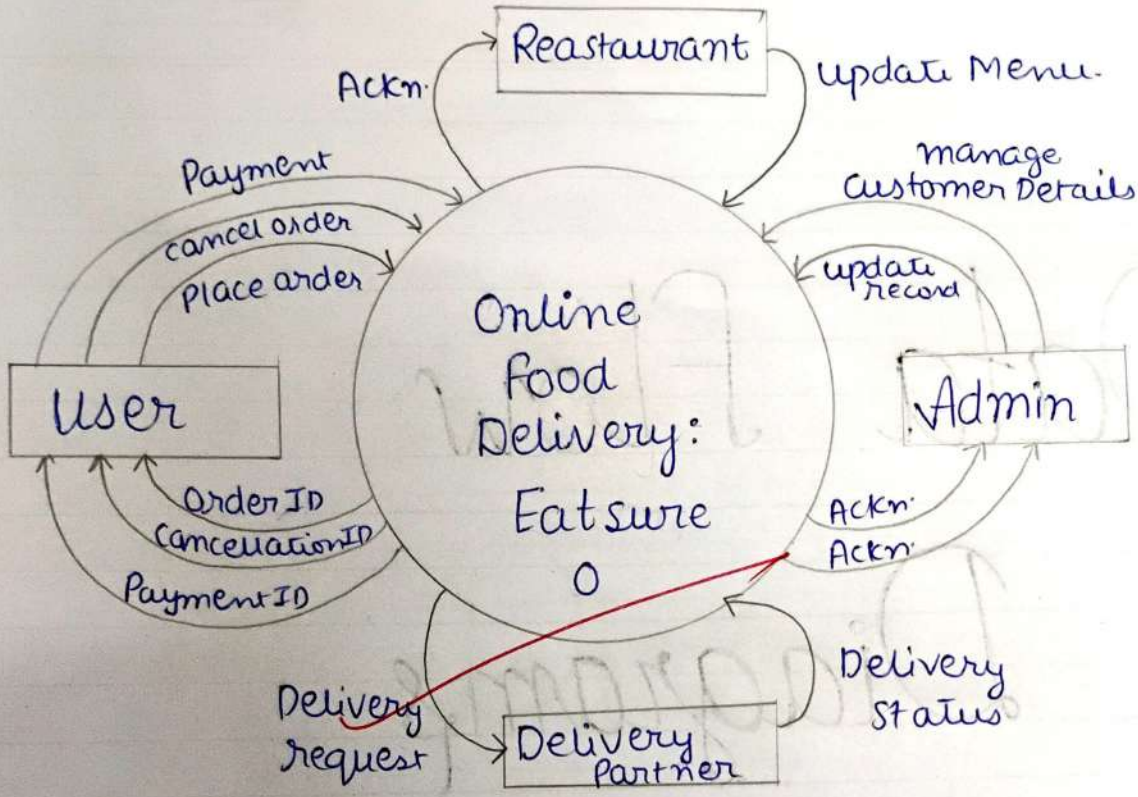
4.2. System Interface

- Integration with payment gateways.
- Integration with GPS Tracking System.
- Database Management System.

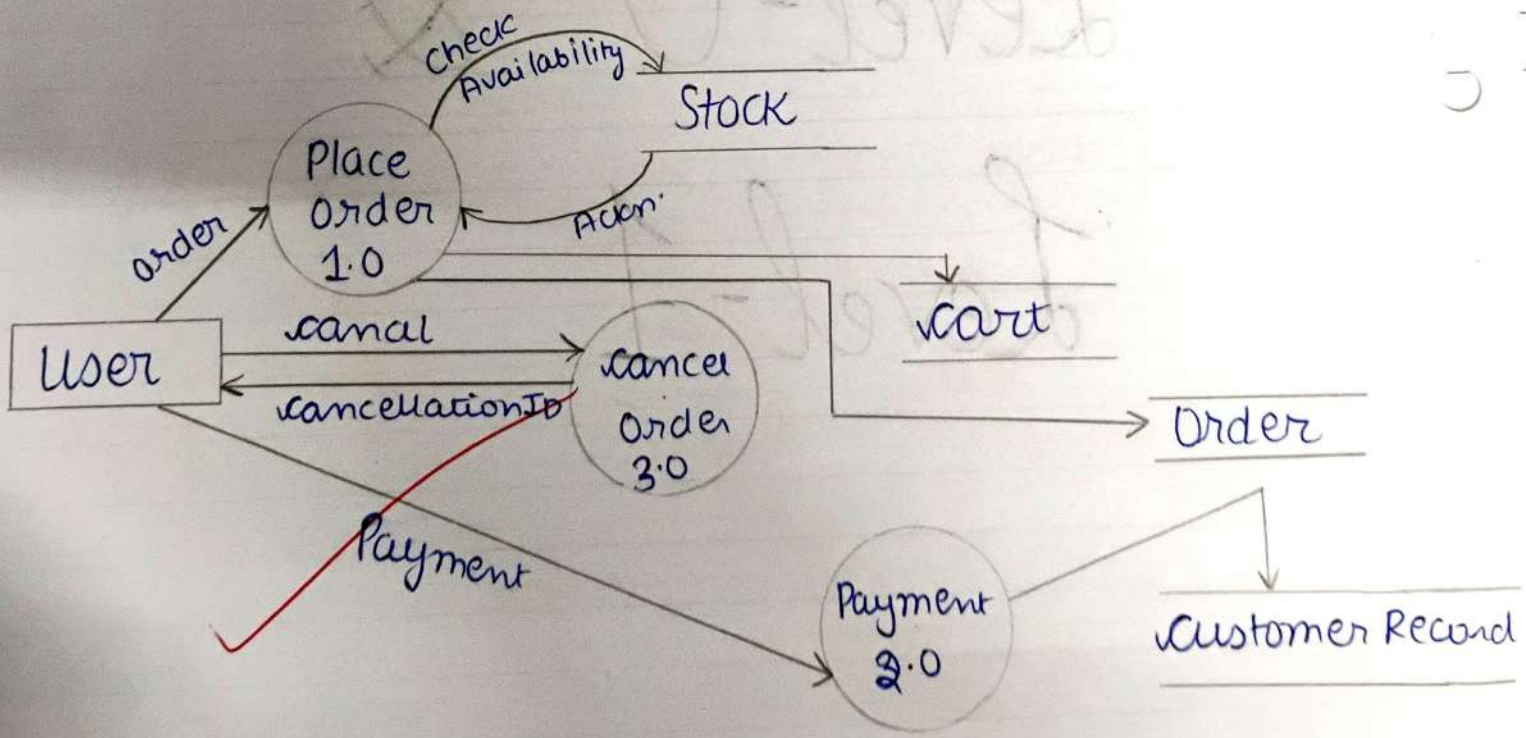
CLASSMATE
Date :
Page :

Data Flow Diagrams Level-0 & Level-1

Level 0 - DFD.



Level 1-DFD

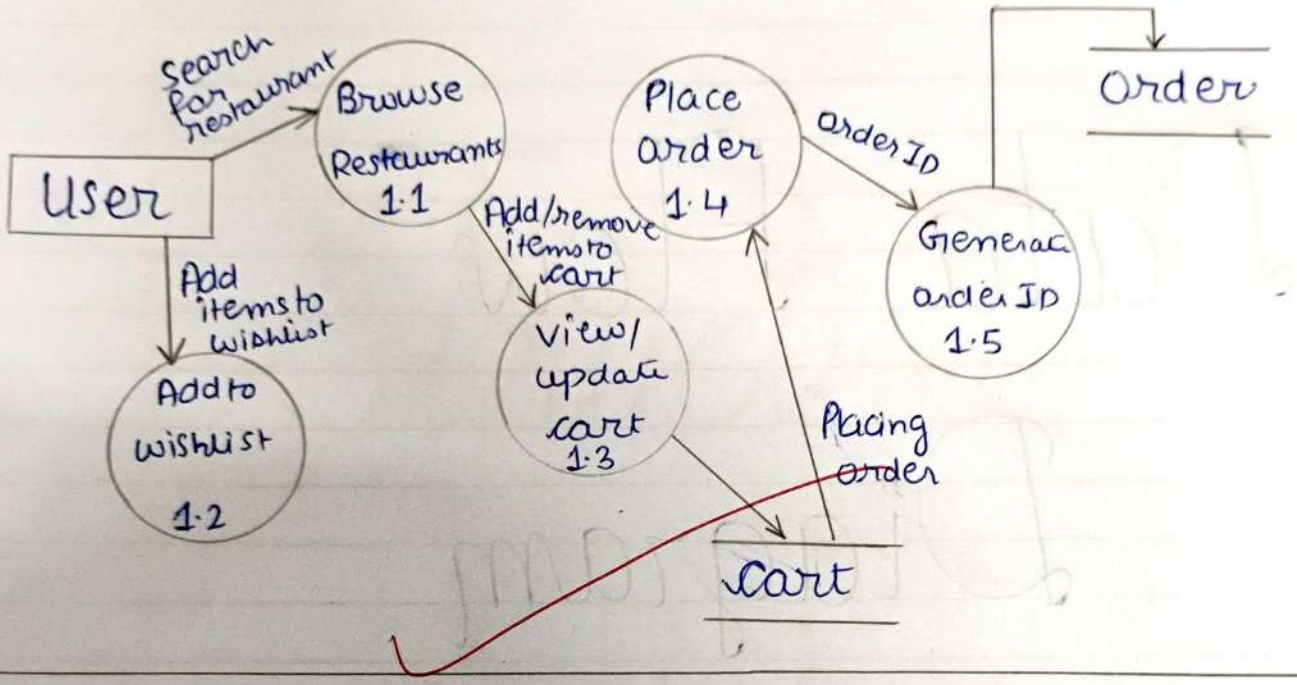


CLASSMATE
Date :
Page :

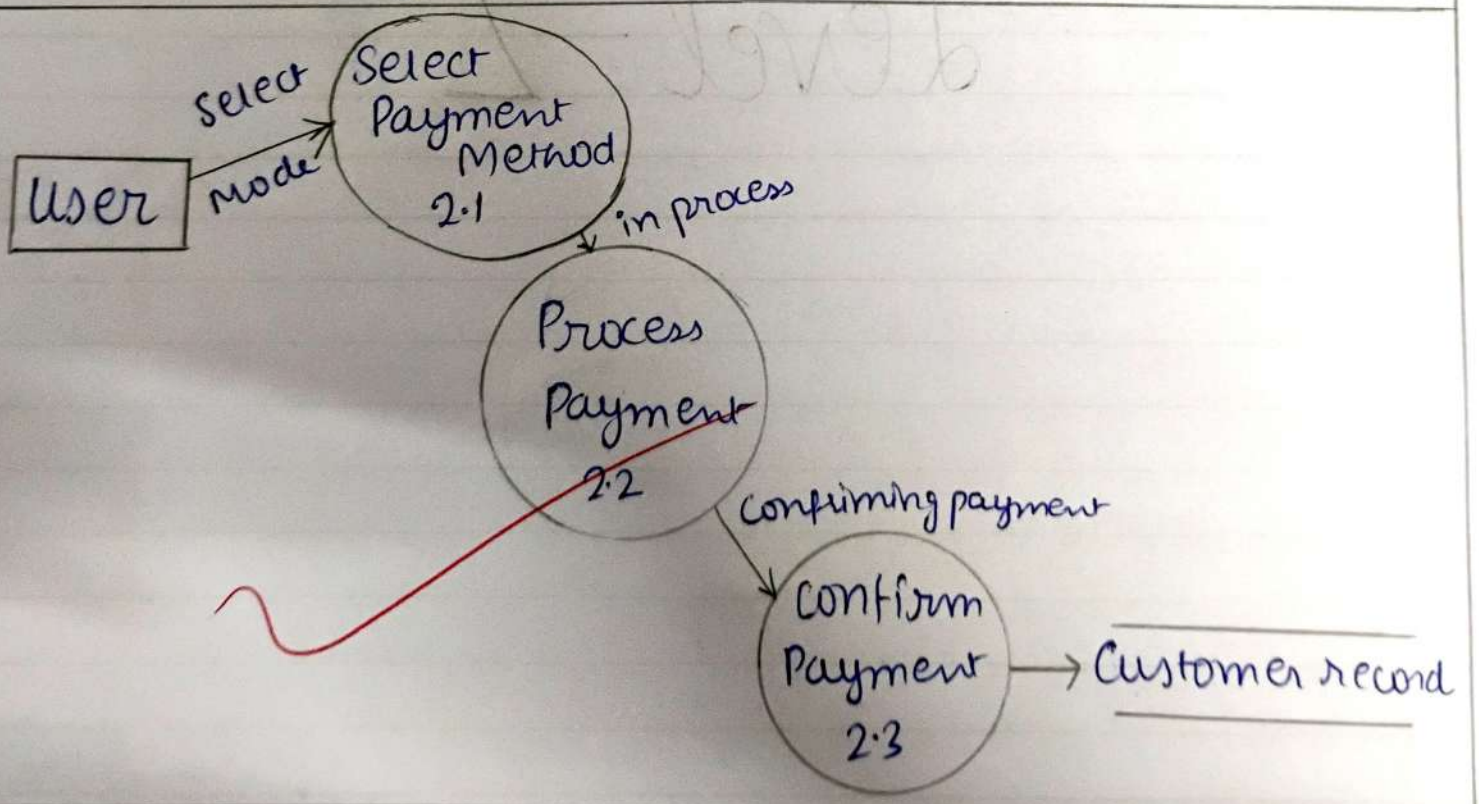
Data Flow Diagram Level - 2

Level-2 DFD.

Process 1:- Place order

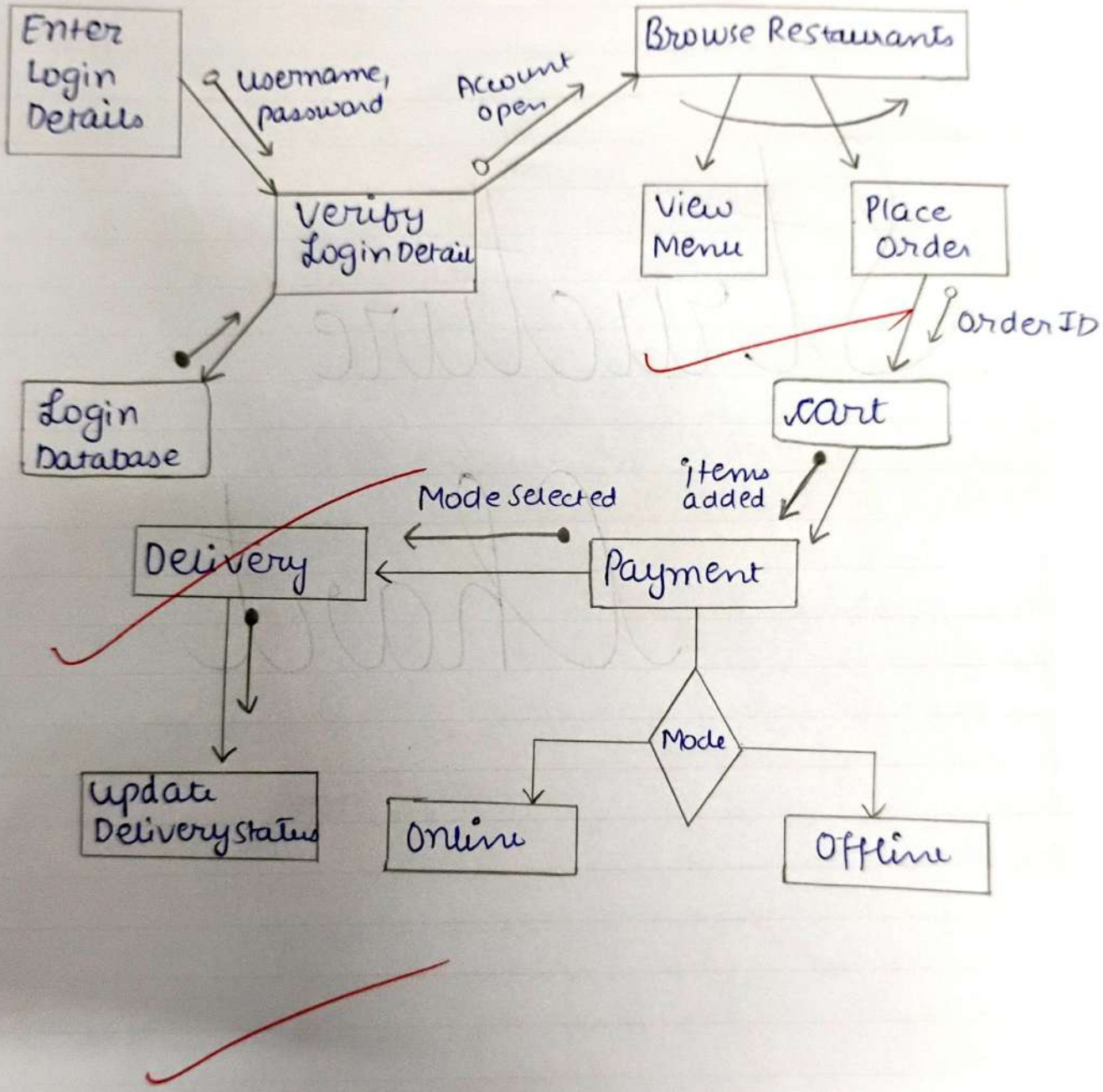


Process 2:- Payment



Structure Chart

Structure Chart

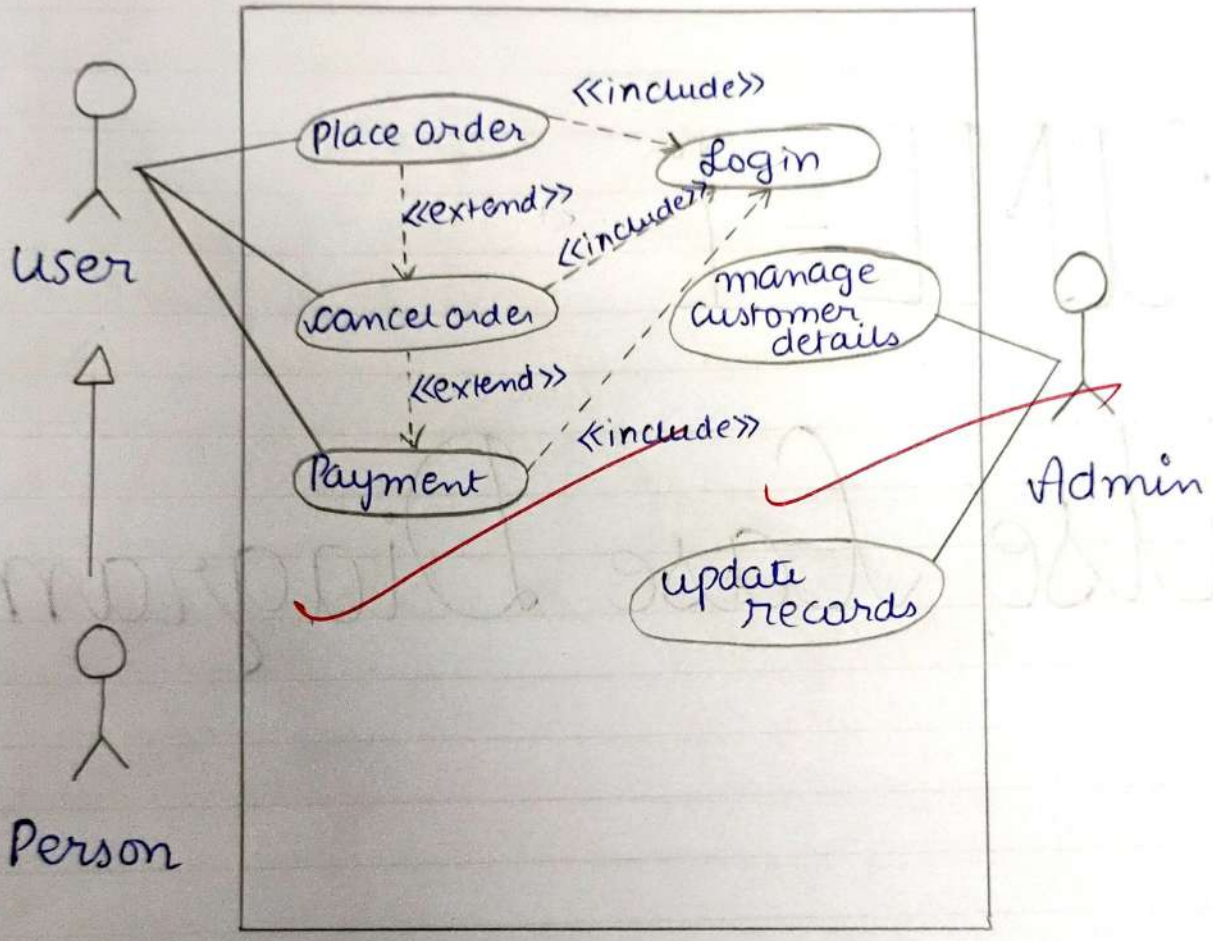


classmate
Date :
Page :

UML-1

Use Case Diagram

Use case Diagram

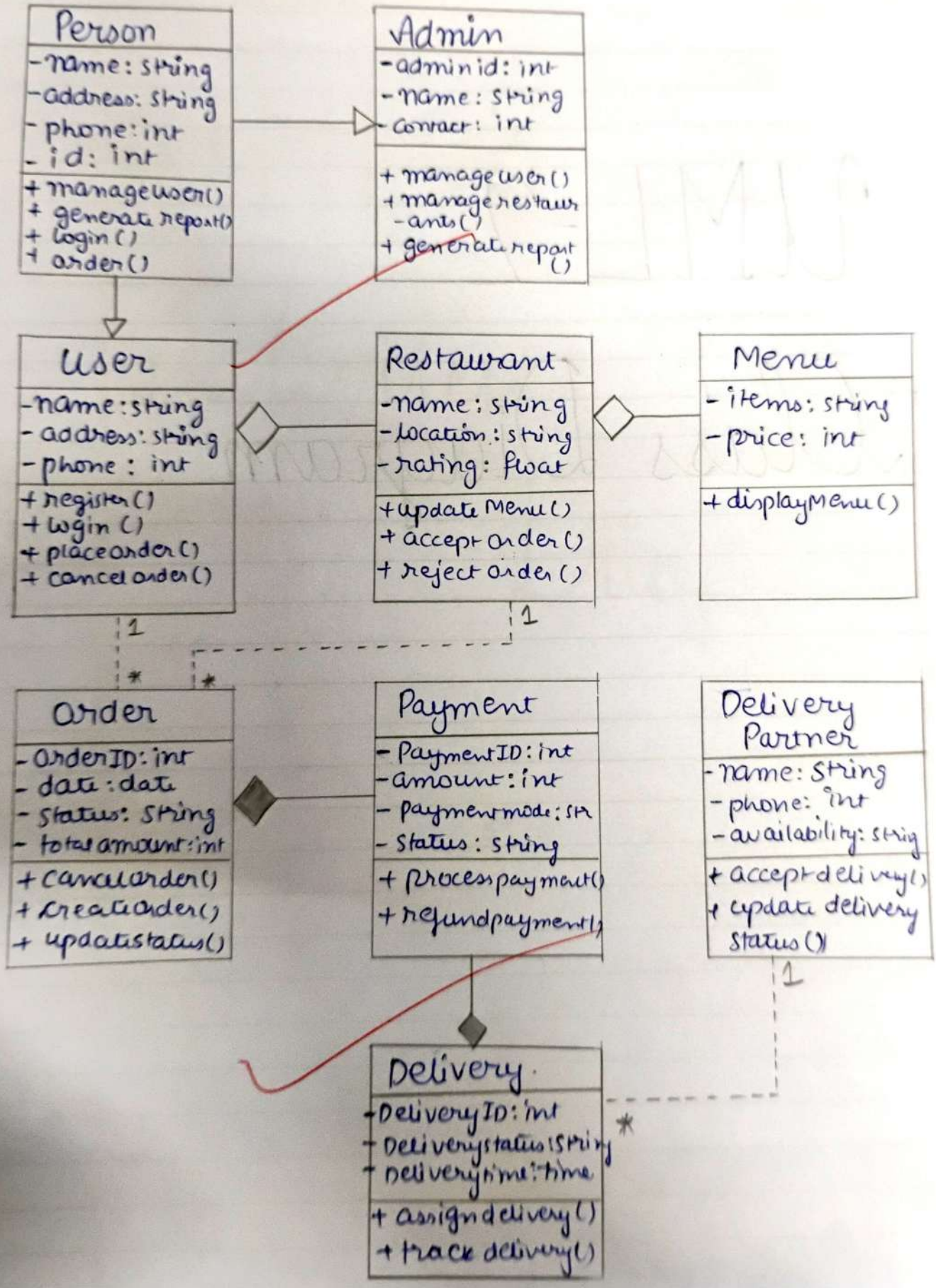


CLASSMATE
Date :
Page :

UML-2

Class Diagram

Class Diagram

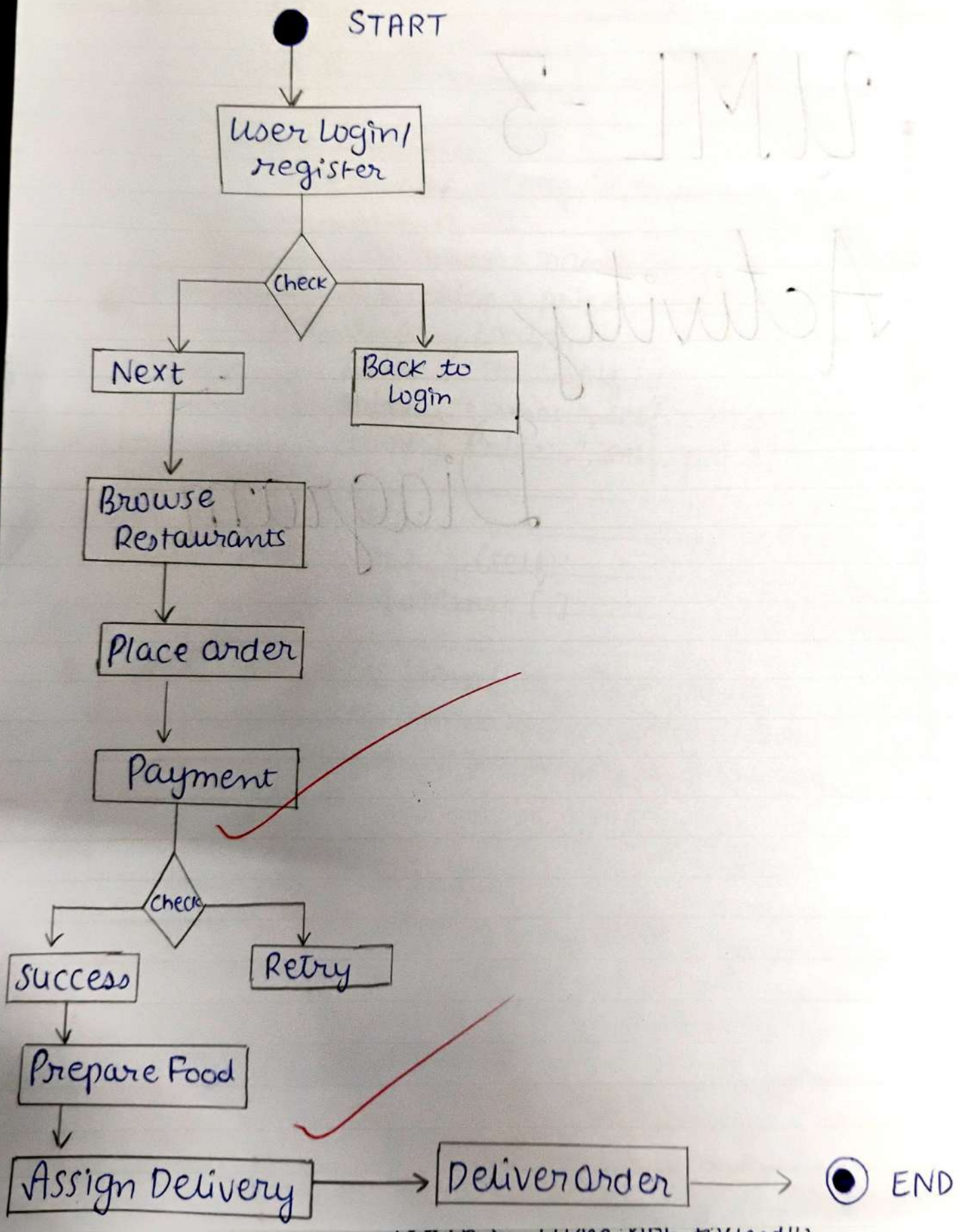


CLASSMATE
Date :
Page :

UML -3

Activity Diagram

Activity Diagram



Test Suite - Eatsure

```
class FoodItem:
    def __init__(self, id, name, price):
        self.id = id
        self.name = name
        self.price = price
    def display(self):
        print("ID: ", self.id)
        print("Name: ", self.name)
        print("Price: ", self.price)

class Order:
    def __init__(self):
        self.items = []

    def add_item(self, food_item):
        self.items.append(food_item)
        print("Item added successfully")

    def remove_item(self, item_id):
        found = False
        for item in self.items:
            if item_id == item.id:
                self.items.remove(item)
                print("Item remove successfully")
                found = True
                break
        if not found:
            print("Item not found")
```

```
def view_order(self):
    if not self.items:
        print("Order is empty")
        return
    total=0
    for item in self.items:
        item.display()
        total+= item.price
    print("Total Bill", total)

Class EatsureApp:
    def __init__(self):
        self.menu= []
        self.order= Order()
    def add_food(self, id, name, price):
        self.menu.append(FoodItem(id, name,
                                  price))

    def show_menu(self)
        print("\n MENU")
        for item in self.menu:
            item.display()
    def search_food(self, keyword):
        found = False
        for item in self.menu:
            if keyword.lower() in item.name.lower(
                item.display()
                found = True
            if not Found:
                print("NO Item found")
```

```
app = EatSureApp()

app.add_food (1, "Burger", 120)
app.add_food (2, "Pizza", 130)
app.add_food (3, "Pasta", 180)

While True:
    print ("1. Show Menu")
    print ("2. Search Food")
    print ("3. Add to Order")
    print ("4. Remove from order")
    print ("5. View order")
    print ("6. Exit")

  Choice = int (input ("Enter choice: "))
if choice == 1:
    app. showMenu()
elif choice == 2:
    keyword = input ("Enter Food name: ")
    app. search_food (keyword)
elif choice == 3:
    id = int (input ("Enter Food ID to add: "))
    found = False
    for item in app. menu:
        if item.id == id:
            app.order. add_item (item)
            found = True
            break
        if not found:
            print (" Food not found!")
```

```
elif choice == 4:
    id = int(input("Enter food id to remove: "))
    app.order.remove_item(id)
elif choice == 5:
    app.order.viewOrder()
elif choice == 6:
    print("Existing")
    break
else:
    print("Invalid choice")
```

Equivalence Class Partitioning

TC	Input Condition	Class Type	Input	Expected Output	Pass/ Fail
1.	Email form -at	Valid	user@gmail.com	Accept email	Pass
2.	Email format	Invalid	usergmail.com	Error: Invalid id	Pass
3.	Password Length	Valid	8+ characters	Login allowed	Pass
4.	Password length	Invalid	3 characters	Error message	Pass
5.	Login Credentials	Valid	Correct email & password	Login success	Pass
6.	Login Credentials	Invaild	Wrong pass -word	Login Fail	Pass
7.	Reastaurant Search	Valid	Pizza	Resaturant shown	Pass
8.	cart quantity	Valid	1-10 Items	Items added to cart	Pass

PREMIUM **** Serial No. **** **** Notes **** **** Date / / ****

TC	Input Condition	Class Type	Input	Expected output	Pass/ Fail
9.	Cart Quantity	Invalid	0 Items	Error message	Pass
10.	Address Input	Invalid	Empty Field	Error: Address req.	Pass
11.	Payment Method	Invalid	Expired Card	Payment Failed	Pass
12.	Order Placement	valid	All Correct details	Order placed Successful	Pass

Boundary Value Analysis

TC	Input Condition	Boundary condition	Input value	Expected output	Pass/ fail
1.	Password length (8-16)	Min Boundary	8 char	Accepted	Pass
2.	Password length	Just below min	7 char	Error message	Pass
3.	Password length	Max Boundary	16 chars	Accepted	Pass
4.	Password length	Just above max	17 chars	Error message	Pass
5.	Cart Quantity (1-10)	Min Boundary	1 item	Added to cart	Pass
6.	Cart Quantity	Max Boundary	10 items	Accepted	Pass
7.	Cart Quantity	Just above max	11 items	Errors	Pass

PREMIUM **** Serial No. **** **** Notes **** **** Date / / ****

TC	Input condition	Boundary	Input value	Expected output	Pass/ fail
8.	Phone number (1-10)	Exact Boundary	10digit	Accepted	Pass
9.	OTP(6digit)	Exact Boundary	6digit	Verified	Pass
10.	Delivery Time (1-60min)	Max Boundary	60min	Order accepted	Pass

Decision Table Testing

TC	Food Selected	Quantity	Payment Method	Expected output	Stautus (P/F)
1.	" " (None)	1	UPI	Error (No Item Selected)	Pass
2.	Burger	0	Card	Error (Invalid quantity)	Pass
3.	Pizza	2	" " (None)	Error (select payment)	Pass
4.	Pasta	1	Cash	Order Placed Successfully	Pass
5.	Burger	3	UPI	Order Placed Successfully	Pass

PREMIUM **** Serial No. **** **** Notes **** **** Date / / ****

Statement Coverage

TC	Title	Content	Expected output	Status (P/F)
1.	Appdaunch	Open Eatsure	Homepage disply	Pass
2.	Successful login	Enter email & password	Login successful	Pass
3.	Failed login	Enter invalid credentials	Error message displayed	Pass
4.	View & SearchResta.	Navigate + Search Burger	Restaurant list & result shown	Pass
5.	Add & remove Item cart	Add item → remove item	Item added/ removed succesful	Pass
6.	Apply coupon	Enter valid coupon code	Discount applied	Pass
7.	Invalid coupon	Enter wrong Coupon code	Error message	Pass
8.	Address Handling	Add valid + invalid address	Saved or error message	Pass
9.	Payment Process	valid & Invalid payment meth -od	Success or failure message	Pass
10.	Place Order	Complete check our process	Order placed Successfully	Pass
11.	Track Order	Open Order tracking	Order status displayed	Pass
12.	logout	Click logout button	User Logged Out	Pass

PREMIUM Serial No. Notes Date / /

Branch Coverage

TC	Module	Branch Condition	Test Data	Expected Output	Status (P/F)
1.	Login	Phone no. Valid? (Yes)	10-digit Valid no.	OTP Sent	Pass
2.	Login	Phone no. Valid? (No)	9-digit invalid	Error: Invalid no.	Pass
3.	OTP Verification	OTP correct? (Yes)	Correct 6-digit OTP	Login Successful	Pass
4.	OTP Verification	OTP correct? (No)	Wrong OTP	Error: Invalid OTP	Pass
5.	OTP Verification	OTP length Valid? (Yes)	6-digit OTP	Accepted	Pass
6.	OTP Verification	OTP length Valid? (No)	5-7 digit OTP	Rejected	Pass
7.	Search	Input empty? (Yes)	" " Empty	Show suggestion	Pass
8.	Search	Input empty? (No)	"Pizza"	Show result	Pass
9.	Cart	Cart empty? (Yes)	0 items	Show "cart is empty"	Pass
10.	Cart	Cart has item? (No)	1+ items	Cart displayed	Pass
11.	Quantity	Within limit? (Yes)	1-10 items	Item added	Pass
12.	Quantity	Above limit? (No)	11+ items	Error: limit exceeded	Pass

PREMIUM **** Serial No. **** **** Notes **** **** Date / / ****

Path Coverage

TC	Title	content	Expected Output	Status (PIF)
1.	Complete Successful Order path	Login → Search → Add to cart → valid address → Successful pay.	Order placed Successful	Pass
2.	Path with Login Failure	Invalid login → Stop process	Error: Login Failed	Pass
3.	Path with Empty cart	Login → Cart empty → Try Checkout	Error: cart is empty	Pass
4.	Path with Invalid address	Login → add item → valid address → Payment failed	Errors: Payment Failed	Pass
5.	Path with Payment Failure	Login → add item → valid add. → Payment failed	Errors: Payment Failure	Pass
6.	Path with Coupon Applied	Login → add item → Apply Valid Coupon → Payment succes	Order placed with discount	Pass
7.	Path with Invalid Coupon	Login → add item → Apply Invalid Coupon	Error: Invalid Coupon	Pass
8.	Path with Item removal	Login → add item → remove item → cart empty	Error: cart is empty	Pass

PREMIUM **** Serial No. **** **** Notes **** **** Date / / ****

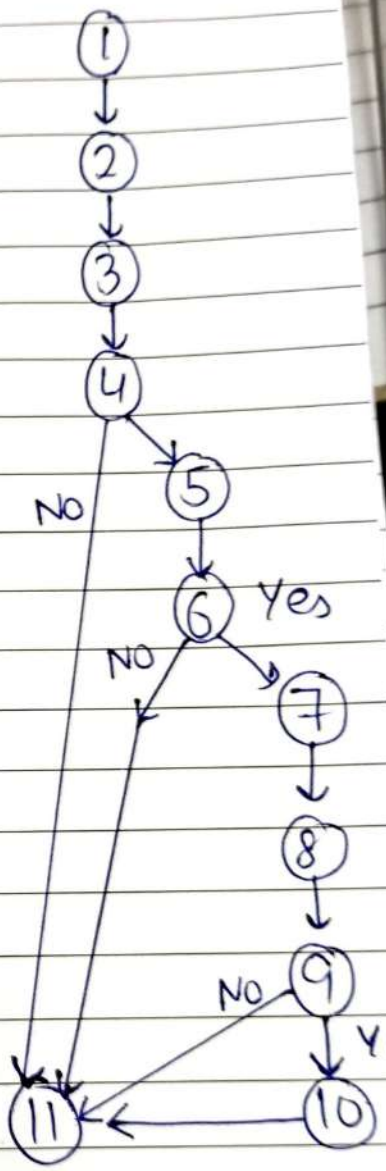
Condition Coverage

TC	Title	Content	Expected Output	Status (P/F)
1.	All Condition true	User logged in = True, Cart not empty = True, Address valid = True, Payment success = True	Order placed Successful	Pass
2.	Login Condition False	User logged in = ~~True~~ False, Cart not empty = True Address valid = True	Error: login Failed	Pass
3.	Cart Condition False	User logged in = True, Cart not empty = ~~True~~, false, Address Valid = True, Payment = True	Error: Cart is empty	Pass
4.	Address Condition False	User logged in = True, Cart not empty = True, Address valid = False, Payment = True	Error: Address Invalid	Pass
5.	Payment Condition False	User logged in = True, Cart not empty = True, Address = True, Payment = false	Error: Payment Invalid	Pass
6.	Multiple condition False	User logged in = True, Cart not empty = False, Address valid = False, Payment = False	Error Displayed	Pass

PREMIUM Serial No. Notes Date

Control flow Diagram

1. Node: Node about place_order()
1. Start
2. Display Menu
3. User select fooditem
4. Check: item selected?
   No → Show "select atleast one item"
5. Add to cart
6. Check: proceed to checkout?
   No → Go to menu
7. Enter delivery details
8. Make payment
9. Check: payment success?
   No → Show Payment Failed
10. Order confirmed
11. End

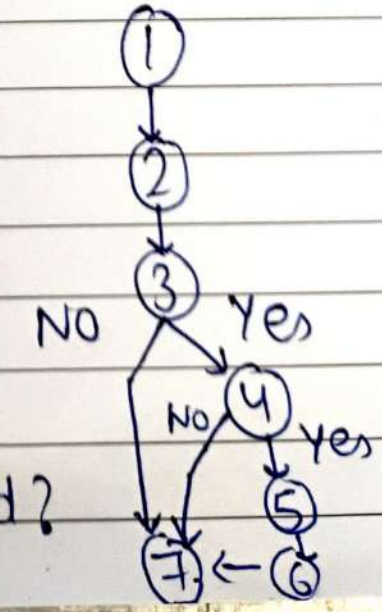


Cyclomatic complexity

CC = No. of decision + 1

CC = 3 + 1 = 4.

2. Node: Cancel_order
1. Start
2. Enter order id
3. Check order exist?
   NO → Print Order Not Found
4. Check order already delivered?
   Yes → Print Cancel Order



5. Cancel Order
6. Refund initiated
7. End

CC = no. of decision + 1
CC = 2 + 1 = 3.